A Fail-Safe Exact Kernel for Rational Sphere–Plane CSG with a Machine-Checked Combinatorial Boundary

Abstract

We present an exact, deterministic boundary kernel for regularized Boolean CSG over rational spheres and planes. The kernel builds arrangements on support surfaces, identifies shared intersection arcs by canonical structural keys, and extracts an oriented boundary complex rather than a mesh. Its combinatorial closure rule is proved sound and complete in Lean 4 for arbitrary half-edge sequences and is independent of half-edge order; the exact comparators on $\mathbb{Q}(\sqrt{d})$ and $\mathbb{Q}(\sqrt{d_1},\sqrt{d_2})$ are proved to be linear orders; and the face-cycle relation of an extracted DCEL is proved to be an equivalence relation. We define a four-way outcome classification (decided-watertight, decided-empty, fail-safe undecided, unsound) and report zero false-watertight outcomes on random and adversarial benchmarks, including kissing spheres, tangent planes, and non-manifold pinches. Rust-to-Lean refinement is supported by mechanical extraction, by exhaustive comparison on 34,462,856 bounded half-edge sequences, by a 1,000-scene Rust-to-Lean agreement run, and by a 2,000-scene outcome run. We state precisely what remains unverified: arrangement construction, global embedding, and containment.

---

1. Introduction

Implicit representations (SDF/CSG) are convenient for design, but converting them to a boundary representation is where robustness usually fails. Floating-point evaluation of `min`/`max` near coincident surfaces produces slivers and non-manifold artefacts; meshing the result hides rather than removes these failures.

This paper takes a different route for a deliberately restricted but practically relevant class: regularized Boolean CSG over **rational spheres and planes**. Instead of sampling a field and meshing it, we compute an exact arrangement on each support surface, select the cells that separate the solid from its complement, and emit an **oriented boundary complex**. All decisions are made in $\mathbb{Q}$, $\mathbb{Q}(\sqrt{d})$, or $\mathbb{Q}(\sqrt{d_1},\sqrt{d_2})$; floating point is used only for display and for conservative broad-phase pruning.

The second theme is **fail-safe verification**. We do not claim to solve arbitrary degeneracies. We claim that the kernel either returns a combinatorially closed boundary complex or reports `Undecided`, and that the combinatorial closure rule is machine-checked. A four-way classification makes the distinction explicit and makes "zero false-watertight outcomes" a measurable property rather than a slogan.

Contributions

1. **Exact arrangement + boundary complex (not a mesh).** Per-support-surface arrangements, canonical arc keys for half-edge identification, detection of `EdgeOverload` / `BowtieVertex`, and Euler characteristic / genus reporting.
2. **Machine-checked combinatorial closure for arbitrary inputs.** `openEdges hs = []` ↔ `checkMatch hs = true` for arbitrary half-edge sequences, plus order-independence (`checkMatch_perm`) and a membership characterization of open edges.
3. **Machine-checked exact comparators.** Both the single-radicand comparator on $\mathbb{Q}(\sqrt{d})$ and the biquadratic comparator on $\mathbb{Q}(\sqrt{d_1},\sqrt{d_2})$ are proved reflexive, antisymmetric, trichotomous, and transitive.
4. **Verified DCEL axioms and cycle partition on extracted data.** `twin` is a fixed-point-free involution; `next` is bijective with an explicit inverse; the face-cycle relation of `next` is an equivalence relation.

5. **Fail-safe evaluation.** A four-way classification and zero false-watertight outcomes on random and adversarial scenes.
6. **Reproducible refinement evidence.** Bounded exhaustive comparison (34,462,856 lists), 1,000-scene Rust↔Lean agreement, a 2,000-scene outcome run, and a `sorryAx` audit.

What we do **not** claim

- A verified arrangement construction (event completeness, face counting).
- Self-intersection-freeness on an exact `Q` vertex mesh, global embedding, or containment.
- Arbitrary degeneracies, general quadrics (treated in a companion paper [12]), or higher-level claims.
- Exactness of the mesh path (it uses floating-point welding and is best-effort).

---

2. Semantics

2.1 Inputs

Primitives are rational spheres `Sphere { c, r }` (`c ∈ Q³`, `r ∈ Q`, `r > 0`) and rational half-spaces `Plane { n, d }` (`n ∈ Q³ \ {0}`, `d ∈ Q`), with interior `n·x ≤ d`. A CSG tree is built from `Const(±1)`, `Prim(i)`, `Union`, `Inter`, and `Diff`. A fixed axis-aligned box is used when bounded clipping is required.

2.2 Regularized Boolean operations

We interpret operations as regularized Boolean operations:

```
A ∪* B := cl(int(A ∪ B)),   A ∩* B := cl(int(A ∩ B)),   A \* B := cl(int(A \ B)).
```

Regularization is not cosmetic. For example, the union of two externally tangent balls, or the intersection of a plane with its opposite half-space, produces lower-dimensional dangling faces under non-regularized semantics. Those artefacts contradict the oriented 3D boundary complex we output. The boundary of a solid `S` is `∂S = ∂cl(int(S))`.

2.3 Face selection

Let `P_i` be a support surface and `F` a 2D arrangement cell on it. We evaluate the exact signs `s_j ∈ {-1, 0, +1}` of all other primitives at a representative point `p ∈ relint(F)`. If any `s_j = 0`, we return `Undecided`. Otherwise we evaluate the CSG tree twice, with `s_i = -1` (inside the support surface) and `s_i = +1` (outside). If the two Boolean values agree, `F` is not on the boundary; if they differ, `F` is a boundary face, with outward orientation determined by the direction of the transition.

 Figure 1: face selection

Figure 1. Face selection on a support surface. The exact CSG value is evaluated on both sides of the support; a cell is on the boundary exactly when the value changes. If any other primitive vanishes at the representative point, the report is fail-safe.

Conditional soundness. If the arrangement is correct and `p` lies in the relative interior of `F`, the selected faces are exactly `∂S`. The proof is short: `int(F)` is connected, each `s_j` has constant sign there, and the sign of the CSG expression changes across `P_i` exactly on boundary cells. The hypothesis "the arrangement is correct" is a premise, not a conclusion; Section 7 states what is and is not verified.

---

3. Exact kernel

 Figure 2: pipeline

Figure 2. Pipeline. Decisions are exact (`Q`, `Q(√d)`); floating point is confined to display and conservative broad-phase pruning. Degeneracies leave the main path through the fail-safe `Undecided` outcome.

3.1 Arrangements and arc keys

For each support surface we compute the 2D arrangement of its intersections with the other surfaces. Spheres contribute circles, planes contribute lines, and plane–plane pairs contribute lines. Intersections are represented in `Q(√d)`; when two different radicands meet on the same surface, the comparator embeds both into `Q(√d₁,√d₂)`.

Each arc receives a **canonical structural key**: the pair of support surfaces, the curve identifier, the canonical lower/upper endpoints, and the arc sector. The key is independent of the observation direction: observing the same shared arc from the other side yields the same key. This removes the need for geometric vertex welding in the combinatorial layer.

3.2 Half-edges and closure

Faces emit directed half-edges `(key, from, to, forward)`. A key is closed when it occurs exactly once in each direction. The Rust function `half_edges_close` implements exactly this count; the Lean function `checkMatch` is its specification. Overloads (more than two half-edges on a key) and bow-tie vertices are reported separately and make the report `Undecided`.

3.3 Euler characteristic and genus

The boundary complex records `V`, `E`, `F`, the number of surface components, and a genus estimate. For a closed orientable surface, `χ = 2C − 2g`; the kernel checks that `χ` is even and at most `2C` before reporting a genus.

Table 1. Representative cases. `W` = decided-watertight, `E` = decided-empty, `U` = fail-safe.

case	faces	χ	outcome
octant (sphere ∩ 3 half-spaces)	4	2	W
lens (two spheres)	2	2	W
hemisphere (sphere ∩ plane)	2	2	W
micro cap	2	2	W
pinch quadrants	4	−4	U (open=10, nm=4)
tangent plane	0	—	U
empty intersection	0	—	E

3.4 Canonical keys and representative points

Keys. A support curve is identified by `EdgeCurve { lo, hi, frag }`: the unordered pair of support-surface identifiers (`lo < hi`) and a fragment index. An arc is identified by `ArcKey { curve, lo, hi, arc }`, where `lo` and `hi` are the canonical endpoint identifiers in ascending order and `arc` selects the sector. For a circle cut by another surface, the two sectors are distinguished by the exact sign of the arc midpoint with respect to the curve's canonical direction; reversing the observation swaps `lo`/`hi` and flips `arc`. For a line, the sector is always `0` and the key reduces to the ordered endpoints. The reversal identities (`arcKey_rev`, `arcKey_eq_of_shared`) are formalized in `S4Arrangement.lean`.

The canonical form is chosen so that two observations of the same shared arc produce the same key without geometric vertex welding. Endpoints are canonicalised by the arrangement vertex identifier, not by floating-point coordinates; coincident rational points therefore receive the same identifier by construction.

Representative point. Each selected face stores a representative point `rep_point` and an inward direction `inward_dir` produced by the arrangement. The exact CSG value is evaluated at `rep_point` with `csg_sign_at`,

which uses `Q/Q(√d)` arithmetic and returns `None` if any primitive evaluates to zero. The face-selection rule of §2.3 is then applied to the resulting sign vector.

Certified interior point (cell identity). For a plane face with representative vertex `v` and inward direction `w`, we search for the smallest `δ = 1/(1000·2^k)` such that the segment `v + t·w`, `0 < t ≤ δ`, meets no boundary curve. The test is exact: a line gives a linear intersection; a circle is decided from the signs at `0` and `δ`, the position of the vertex, and the discriminant. The first such `δ` certifies that the step from `v` along `w` crosses no registered boundary curve, so `v + δ·w` lies in the relative interior of a single cell in the `w` direction. This discharges the interior-point premise `p ∈ relint(F)` of §2.3 on the certified faces, **assuming arrangement correctness and a valid inward direction** `w`; arrangement correctness remains an unverified hypothesis.

For a sphere face we use the rational geodesic `q(S) = ((1−cS²)v + 2Sw)/(1+cS²)` with `c = |w|²`, which stays on the unit sphere because `w` is tangent at `v` (`v·w = 0`). Substituting into a boundary circle `n·p = e` gives a quadratic `−c(n·v+e)S² + 2(n·w)S + (n·v−e)`; the same sign/vertex/discriminant test finds the first `S = 1/(1000·2^k)` with no crossing. The resulting `q(S)` is a certified relative-interior point of the sphere face.

What is certified. The checker is a runtime exact certificate: for each face it either returns an interior point with a certified step, or fails (and the face is not certified). On all 578 decided-watertight fuzz scenes, **1,050/1,050 sphere faces** and **386/388 plane faces** are certified at the face level; on the two remaining plane faces (one degenerate zero-inward face, one unbounded face), the runtime certificate checker returns `None` and reports **fail-safe at the certificate layer**. This checker is a post-hoc audit of extracted faces, not a gate inside `classify`; therefore the scene-level outcome remains decided-watertight. The **linear case** of the step test (a line, or a boundary circle through the representative vertex) is also proved in Lean (`S4Relint.linear_no_zero`). The checker as a whole is not a Lean theorem about arbitrary inputs; it discharges only the interior-point premise of §2.3 on the certified faces, and arrangement correctness remains a hypothesis.

---

4. Machine-checked core

All theorems below are in Lean 4. The Mathlib-free development is the default; the Mathlib-dependent files are checked separately. No theorem depends on `sorryAx`.

Methodology. Each theorem is checked in one of two regimes: (i) arbitrary-input theorems in the Mathlib-free core, with an axiom audit (`Axioms.lean`); or (ii) Mathlib-dependent order and geometry proofs, kept in separate files so the fast path stays dependency-free. Extracted-data checks are `decide`-checked against mechanically dumped Rust data and are never presented as arbitrary-input theorems. Table 7 and Appendix A use the same classification.

4.1 Closure kernel (arbitrary inputs)

```
theorem openEdges_eq_nil_iff (hs : List HE) :
    openEdges hs = [] ↔ checkMatch hs = true
```

`HE` is a half-edge with a canonical arc key and a direction. The theorem holds for **any** list, not only for extracted data. Additional results:

- `checkMatch_perm` — closure is invariant under permutation of the half-edge list.
- `closureOk_perm`, `openEdges_nil_perm` — the same for the public closure predicate.
- `mem_openEdges_iff` — a key is open iff its forward or backward count is not one.
- `checkMatch_total_two` — accepted inputs have exactly two half-edges per key.

4.2 Exact comparator

The comparator is defined on integer pairs representing `p + r√d`. We prove

```
theorem cmpQuad_trans (h1 : cmpQuad d a b ≠ .gt) (h2 : cmpQuad d b c ≠ .gt) :
    cmpQuad d a c ≠ .gt
```

together with reflexivity, antisymmetry, trichotomy, and the strict version. The proof embeds `p + r√d` into `ℝ`, shows the sign function agrees with the real sign, and uses the linear order of `ℝ`. It uses Mathlib (`Classical.choice`), but no `sorryAx`.

Coverage. The Lean development proves the order axioms for both layers: the single-radicand comparator on `Q(√d)` (integer and rational coefficients) and the biquadratic comparator on `Q(√d₁,√d₂)` used when two radicands meet (`S4BiQuadMathlib.lean`). The latter mirrors `sign_p_plus_q_sqrt`: it compares `p²` and `q²d₂` in `Q(√d₁)` and uses that comparison to decide the sign of `p + q√d₂`. The radicands are natural numbers (`d, d₁, d₂ : ℕ`); the biquadratic sign theorem assumes `d₂ > 0`. Table 7 records the result.

4.3 DCEL on extracted data

From a watertight report we extract `twin`, `next`, and the inverse `inv`. The Lean checker `S4DCEL.ok` verifies, pointwise:

- `twin (twin i) = i` and `twin i ≠ i`;
- `next (inv i) = i` and `inv (next i) = i` (hence `next` is bijective);
- all indices are in range;
- the number of half-edges is even (`okAll_even`).

Four representative scenes pass:

Table 2. Extracted DCEL checks.

scene	half-edges	faces	checks
octant	12	4	twin involution, no fixed point, next bijection
lens	4	2	same
hemisphere	4	2	same
micro cap	4	2	same

On the 578 decided-watertight fuzz scenes, **374 have a non-empty half-edge cycle structure and all 374 pass the same checks** (Rust-side, same axioms as the Lean checker). Of these, **338 are complete** (every face has a vertex-subdivided boundary); on all 338 the number of `next` cycles equals the number of faces and the recomputed Euler characteristic `V − E + F` equals the reported `euler`. The other 36 contain some zero-vertex faces and are treated as partial DCELs. The remaining 204 scenes consist solely of isolated circular boundaries with no vertex-subdivided loops; the current half-edge representation has no DCEL for them, so they are reported as **not applicable**, not as failures. Four representative scenes are additionally checked in Lean (`decide`).

On the Mathlib side we build `nextPerm : Equiv.Perm (Fin n)` from the extracted data and prove

```
theorem sameCycle_equivalence : Equivalence (nextPerm h).SameCycle
```

so the face cycles of `next` partition the half-edges. Face **counting** and the Euler identity are not yet formalized.

4.4 Outcome classification

`S4Outcome.classify` mirrors the Rust classification. Its W/E/U conditions are proved by exhaustive case analysis on Booleans, and

```
theorem decided_watertight_closure (h : classify ... = .decidedWatertight) :
    closureOk hs = true
```

connects a decided-watertight outcome to the verified closure kernel. `unsound` is deliberately not part of the classifier; it requires an independent oracle and is counted in tests.

---

5. Rust-to-Lean refinement evidence

The Rust closure kernel is a pure function `half_edges_close` over `(key, from, to, forward)` tuples; it ignores `from`/`to` and counts forward/backward occurrences per key. This is the same algorithm as Lean's `checkMatch`. We support the correspondence in three ways.

1. **Mechanical extraction.** Representative scenes are dumped from Rust to Lean (`S4Data_generated.lean`) and checked against the Lean predicates (`S4Checks.lean`).
2. **Bounded exhaustive comparison.** On a small alphabet (3 keys × 2 directions, lengths 0–9; 2 keys × 2 directions, lengths 0–12) we compare the Rust function with an independent reference mirroring the Lean definition: **34,462,856 lists, zero mismatches** (about 8 s).
3. **Full-fuzz agreement.** All 1,000 fuzz scenes are dumped and checked with `checkMatch`: **1,000/1,000** (Lean about 9.3 s). A separate 2,000-scene Rust run is used for the outcome counts in Table 3 and Table 4.

This is not an arbitrary-length refinement proof. It closes the transcription gap empirically and leaves a formal refinement tool (e.g., Aeneas) to future work.

---

6. Evaluation

6.1 Outcome classification

Table 3. Random CSG scenes (fixed seeds):

N	decided-watertight	decided-empty	fail-safe	unsound
400	238	60	102	**0**
1000	578	189	233	**0**
2000	1170	354	476	**0**


Figure 3: outcome counts

Figure 3. Outcome counts. The fail-safe column is a feature, not a success rate; `unsound` is zero in all three runs.

Exact-ray oracle. For each watertight scene we cast 16 rays `o + t d` with `o, d ∈ Q³` from fixed-seed origins and directions. The oracle first requires `csg_sign_at(o) ≠ 0`; it then computes all intersections with the primitives exactly, sorts the parameters `t`, and counts the sign changes of the CSG expression along the ray. This count is compared with the number of boundary intersections of the extracted complex on the same ray. Rays whose origin lies on a primitive, or whose intersection set is degenerate (tangent, through-edge, duplicate `t`), are skipped. A mismatch is counted as `unsound`. The oracle is a **consistency check** (an empirical soundness audit) on sampled rays, not a proof of soundness; `F_unsound = 0` therefore means "no contradiction found on these rays", and the paper does not claim more.

The table reports raw counts, not a coverage claim: the fail-safe column is a feature, not a success rate.

Undecided breakdown. Table 4 classifies the fail-safe scenes by the first applicable reason, in the order singular contact → non-manifold → open edges → local orientation → no faces → other.

Table 4. Primary reason for fail-safe outcomes.

reason	N = 1,000	N = 2,000
singular contact	98	196
non-manifold	10	27
open edges	120	241
local orientation	3	8
no faces (not decided empty)	1	2
other	1	2
total	**233**	**476**

6.2 Adversarial degeneracies

The eight adversarial categories are kissing spheres external/internal, tangent plane, coplanar opposite half-spaces, coaxial circles, pinch vertices, micro cap, and empty intersection. `W = 2, E = 1, U = 5, unsound = 0`. The five fail-safe outcomes are the intended result for non-manifold contacts and tangencies.

 Figure 4: adversarial categories

Figure 4. Adversarial categories and their outcomes. `pinch quadrants` *reports open edges and non-manifold vertices; it is classified as fail-safe, not rejected silently.*

Table 5. Adversarial categories and the primary fail-safe reason (priority: singular → non-manifold → open edges → other).

category	outcome	primary reason	faces / open / non-manifold
kissing, external	fail-safe	singular	0 / 0 / 0
kissing, internal	fail-safe	singular	0 / 0 / 0
tangent plane	fail-safe	singular	0 / 0 / 0
coplanar opposite	fail-safe	other	2 / 0 / 0
coaxial circles	decided-watertight	—	4 / 0 / 0
pinch quadrants	fail-safe	non-manifold	4 / 10 / 4
micro cap	decided-watertight	—	2 / 0 / 0
empty intersection	decided-empty	—	0 / 0 / 0

6.3 Scaling

Table 6. Left-deep CSG trees over `N` primitives (`arrange_scene` + `boundary_report`, release build). Median and interquartile range over fixed seeds (5 seeds for N ≤ 16, 3 for N = 32):

N	seeds	median	IQR
4	5	5 ms	3–7 ms
8	5	27 ms	9–106 ms
16	5	1,219 ms	593–1,241 ms
32	3	35,621 ms	15,817–35,621 ms

For N = 32 (three seeds), q1 and q3 are the first and third sorted samples; the median is the second, so it can coincide with q3. Timings are machine-dependent; the table records one run.

 Figure 5: scaling

Figure 5. Scaling (log-log): median and interquartile range (5 seeds for N ≤ 16, 3 for N = 32). The `N = 32` case is expensive and excluded from the default test suite.

The spread between seeds is large, especially at N = 8 and N = 32, so `N` alone is not a good predictor. The cost tracks the number of arrangement cells and half-edges produced by the randomly chosen primitive mix. For N = 32 the median is about 30× the N = 16 median; the dominant factor is the exact-arithmetic arrangement size (faces and half-edges), not the CSG tree depth, which is at most `N`. We therefore report the distribution rather than a single timing, and do not claim an asymptotic law.

6.4 What is verified vs. tested

Table 7. What is verified and what is not. "Proof (Lean)" means a Lean theorem for the stated scope (arbitrary or extracted, as marked); "Exhaustive" is a bounded test; "Fuzz" is a fixed-seed run.

Property	Proof (Lean)	Exhaustive (bounded)	Fuzz	Not verified
Combinatorial closure	✅	✅ 34 M	✅ 1000	—
Order independence	✅	—	—	—
Comparator on `Q(√d)` (integer)	✅ (Mathlib)	—	—	—
Comparator on `Q(√d)` (rational)	✅ (Mathlib)	—	—	—
Comparator on `Q(√d₁,√d₂)`	✅ (Mathlib)	✅ 81 coeffs × 3 bases	✅ benchmarks	—
DCEL axioms	—	—	✅ 4 scenes (Lean) + 374/578 W fuzz (204 loop-only; conditional on `okAll`)	construction
DCEL face count / Euler (runtime exact)	—	—	✅ 338/374 complete (36 partial)	general theorem
Cell identity (runtime exact, plane + sphere)	—	—	✅ 1,050/1,050 sphere, 386/388 plane (per-face certificate)	general proof
Step test (linear case)	✅ (Mathlib)	—	—	quadratic/geodesic
Face-cycle partition	✅ (Mathlib, extracted; conditional on `okAll`)	—	—	general counting proof
Global embedding / containment	❌	❌	—	all
Arrangement construction	❌	—	—	all

6.5 Comparison with CGAL

We compared the 2D circle-arrangement component against CGAL 5.6.1's `Arrangement_2` through an independent oracle executable (`tests/cgal_compare.rs`, `SHIKISAI_CGAL_ORACLE`). On non-degenerate inputs, face counts and Euler characteristics agree exactly. CGAL splits curves and vertices at tangencies, so raw `V`/`E` counts differ; on degenerate inputs we report representation-dependent differences rather than claiming a mismatch. This comparison is **2D only**; it does not compare the 3D sphere/plane kernel, and no performance claim is made.

Table 8. 2D circle arrangement: our counts vs CGAL 5.6.1. `χ = V − E + F`.

case	ours (V,E,F)/χ	CGAL (V,E,F)/χ	F	χ
crossing	(2,4,4)/2	(6,8,4)/2	=	=
Venn (3 circles)	(6,12,8)/2	(12,18,8)/2	=	=
disjoint	(2,2,3)/3	(4,4,3)/3	=	=
isolated	(1,1,2)/2	(2,2,2)/2	=	=
external tangency	(1,2,3)/2	(3,4,3)/2	=	=
internal tangency	(1,2,3)/2	(4,4,3)/3	=	≠
triple point	(4,9,7)/2	(6,10,6)/2	≠	=

The four non-degenerate cases are the ones where we assert equality. The degenerate rows document how the two representations differ; in particular, internal tangency gives CGAL `χ = 3` under its local vertex splitting, while our combinatorial self-consistency check gives `χ = 1 + C = 2`.

A randomized diagnostic over 100 generated cases (proper crossings, centers outside each other) agreed on 38 of the 40 non-skipped cases. The two disagreements are **oracle-side**: the external executable's seam removal changes its face count and violates its own Euler identity for those inputs. We therefore do not use the randomized run as evidence and keep the four controlled cases as the quantitative claim.

---

7. Limitations

- **Arrangement construction is not formalized.** The face-selection soundness statement is conditional on the arrangement being correct. Event completeness and face counting are open.
- **Embedding and containment are not verified.** Exact tri-tri classification is available as an opt-in audit on **finite f64 inputs**; it is not an exactness statement about curved surfaces. There is no exact `Q`-vertex mesh path yet, and no global containment check.
- **Refinement is empirical.** The Lean closure theorem is arbitrary-input, but the Rust-to-Lean correspondence is supported by extraction, bounded exhaustive testing, and fuzz, not by a refinement proof.
- **Coverage is limited.** The kernel returns `Undecided` on many random scenes, and on isolated disk faces the current half-edge extraction cannot build a DCEL.
- **The mesh path is best-effort.** `mesh_boundary_clipped_csg` welds vertices on a floating grid (`1e-9`); exactness claims apply to the combinatorial boundary, not to this mesh.
- **Cell identity is certified per face at runtime.** For plane and sphere faces the checker finds a certified interior point with a step, or fails; a Lean theorem for arbitrary inputs is future work.
- **General quadrics and arbitrary degeneracies are out of scope.** The former are treated in a companion paper [12]; the latter are handled by returning `Undecided` rather than by a complete classification.

---

8. Related work

Regularized solid modeling and boundary evaluation. Requicha and Voelcker introduced the regularized-set framework and the boundary-evaluation view of CSG [1, 2]; Tilove's set-membership classification [3] is the conceptual ancestor of our cell-selection rule. We use the same regularization but restrict the primitive class to rational spheres and planes, which makes every predicate exact.

Exact geometric computation. Shewchuk's adaptive predicates [4] are the standard approach to robustness for floating-point input; our approach differs in that the input and the predicates are rational, so no adaptive filter is needed in the combinatorial core. The CGAL arrangement package [5] is the mature implementation of exact 2D arrangements; our contribution is not a new arrangement algorithm but a

machine-checked combinatorial layer and a fail-safe outcome discipline on top of a restricted arrangement kernel.

Curved exact modeling. For curved solids, the CGAL 3D spherical kernel [9] provides exact spherical primitives and arcs, Nef polyhedra [10] support Boolean operations in full generality including non-manifold cells, and ESOLID [11] performs exact boundary evaluation for low-degree curved solids. Our contribution is not a new exact boundary evaluator; it is a machine-checked combinatorial core and a fail-safe outcome discipline over a restricted rational sphere/plane class.

Positioning. The closest systems differ on where verification sits. CGAL's 3D spherical kernel and Nef polyhedra provide exact geometric primitives and Boolean operations, but their correctness is established by implementation and testing rather than by a machine-checked combinatorial kernel. ESOLID performs exact boundary evaluation for low-degree curved solids with exact arithmetic, but does not expose a verified closure rule or a fail-safe outcome classification. This work is complementary: it targets a narrower primitive class (rational spheres and planes) and invests in a Lean-checked combinatorial core, a proved comparator, and an explicit W/E/U/F discipline; it does not claim to replace these systems for general curved modeling.

Machine-checked mathematics and refinement. Lean 4 [6] and Mathlib [7] provide the proof infrastructure; the comparator proof uses Mathlib's linear order on `ℝ`. Aeneas [8] is the natural candidate for replacing our bounded refinement evidence with a proof; we deliberately leave that to future work.

Quadric arrangements. A companion paper [12] treats general quadrics; the present paper is restricted to spheres and planes and does not claim results for quadrics.

---

9. Reproducibility

The reported counts and Lean-check outcomes are recorded as data in this paper/data artifact (`data/artifacts/test-vectors/` and `SHA256SUMS`). The research kernel, test harness, extractors, and Lean sources are not included in this artifact. Wall-clock times depend on the host and are indicative, not bit-reproducible.

9.1 Code and data availability

Code. The S4 kernel, tests, extractors, and Lean proofs are not included in this paper/data artifact. They are not distributed here. No code license is granted by this artifact.

Product boundary. The Axiom product QA core, MCP server, CLI, and CA signing service are separate from this paper and are not included in this artifact. Their licensing is separate and is not determined by this artifact. No proprietary A' source or CA signing key is included. None of the S4 results depend on the product.

Data. All benchmark inputs are synthetic and fixed-seed. No proprietary, personal, or third-party restricted data are used. The numeric outcomes are recorded in `data/artifacts/test-vectors/`. The optional CGAL comparison uses an external executable and is not used for any paper claim.

Certificate and verification. S4's runtime certificate is the per-face `relint_certified` construction together with `decided_watertight_closure`; the Lean statement `decided_watertight_closure` is in `S4Outcome.lean`. The certificate construction is described in this paper; its implementation is not included in this artifact. The Axiom-signed certificate schema is open. Axiom-signed certificates are not used as S4 evidence.

License. Paper text: CC BY 4.0. Artifact data: CC BY 4.0. Third-party tools keep their own licenses. No research code, Lean source, product source, MCP server, CLI, or CA signing key is included in this artifact.

Reproducibility scope. The 34,462,856 bounded half-edge enumeration, the N=400/1000/2000 fuzz counts, the adversarial suite, and the Lean axiom checks are recorded as data in this artifact. Wall-clock times depend on the host and are indicative, not bit-reproducible. The optional CGAL comparison is environment dependent and is not a claim. No S4 count or theorem depends on the Axiom product QA core.

Test vectors are included under `data/artifacts/test-vectors/` in this paper/data artifact.

---

10. Conclusion

For rational sphere–plane CSG, a small combinatorial core can be verified while the geometric construction remains conventional. We have shown that this split is useful in practice: the closure rule is proved for arbitrary half-edge sequences, the comparator is a proved linear order, and the extracted DCEL passes verified structural checks; meanwhile the kernel fails safely on degeneracies instead of producing false watertight outputs. The remaining work is clearly delimited: verify the arrangement construction, add an exact `Q`-mesh path together with embedding and containment, and replace bounded refinement evidence by a formal Rust-to-Lean refinement proof.

Appendix A. Theorem inventory

Table A1. Lean theorems used in the paper. “Arbitrary” means proved for all inputs of the stated model; “extracted” means checked on mechanically extracted data.

theorem	file	scope	axioms
`openEdges_eq_nil_iff`	`S4Core`	arbitrary half-edge lists	propext, Quot.sound
`checkMatch_perm`	`S4Core`	arbitrary	+ Classical.choice
`closureOk_eq_checkMatch`	`S4Core`	definitional	none
`mem_openEdges_iff`	`S4Core`	arbitrary	+ Classical.choice
`checkMatch_total_two`	`S4Core`	arbitrary	propext, Quot.sound
`cmpQuad_refl`	`S4Comparator`	arbitrary (`Int`)	propext
`cmpQuad_antisymm`	`S4Comparator`	arbitrary (`Int`)	propext, Classical.choice, Quot.sound
`cmpQuad_trichotomy`	`S4Comparator`	arbitrary (`Int`)	propext
`arcKey_rev`, `arcKey_eq_of_shared`	`S4Arrangement`	arbitrary	propext
`cmpQuad_trans`, `cmpQuad_lt_trans`	`S4ComparatorMathlib`	arbitrary	+ Classical.choice
`signQuadRat_spec`	`S4BiQuadMathlib`	arbitrary rationals	+ Classical.choice
`signPPlusQSqrt_spec`	`S4BiQuadMathlib`	arbitrary	+ Classical.choice
`signBiQuad_spec`, `cmpBiQuad_refl`, `cmpBiQuad_trans`, `cmpBiQuad_antisymm`, `cmpBiQuad_trichotomy`	`S4BiQuadMathlib`	arbitrary	+ Classical.choice
`twin_involutive`, `next_bijective`	`S4DCEL`	extracted	propext, Quot.sound
`okAll_even`	`S4DCEL`	arbitrary lists	propext, Quot.sound
`sameCycle_equivalence`	`S4DCELMathlib`	extracted	+ Classical.choice
`decided_watertight_closure`	`S4Outcome`	arbitrary `Bool`	propext, Quot.sound
`classify_exclusive`	`S4Outcome`	arbitrary `Bool`	none
`linear_no_zero`	`S4RelintMathlib`	arbitrary reals	+ Classical.choice

The axiom audit prints the axioms of the main theorems and rejects any occurrence of `sorryAx`; the table above is the audited result.

A.1 Proof sketches

Combinatorial closure. `checkMatch` checks, for every key in `uniqKeys hs`, that the forward and backward counts are both one. `openEdges` is `uniqKeys hs` filtered by the negation of that predicate, so the filter is empty exactly when the predicate holds everywhere. The proof is list induction on the filtered list (`filter_not_eq_nil_iff_all`), for an arbitrary half-edge list.

Order independence. Counts are `filter` lengths; a permutation preserves the filtered multiset (`List.Perm.filter`) and its length. The key set is `eraseDups` of the mapped key list, and `all` is invariant under `eraseDups` because duplicates repeat the same predicate.

Single-radicand order. `signQuad` is shown to agree with `Real.sign` of `a + b√d` by case analysis on the signs of `a`, `b` and on `a²` versus `b²d`; the latter is decided exactly. Transitivity follows from the linear order of $\mathbb{R}$.

Biquadratic order. $p + q\sqrt{d_2}$ is decided by the sign of p, the sign of q, and the comparison of p^2 with $q^2 d_2$ in $\mathbb{Q}(\sqrt{d_1})$. The sign specification plus the real order of $\mathbb{R}$ gives transitivity; a rational-coefficient version is used inside the algorithm.

DCEL cycle partition. From `okAll`, `next` is bijective with an explicit inverse, so `nextPerm` is a permutation of `Fin n`; `Equiv.Perm.SameCycle` is then reflexive, symmetric, and transitive.

Linear step test. A linear function with the same nonzero sign at both endpoints is monotone; an interior zero would force a sign change, a contradiction.

Appendix B. Artifact manifest

The paper/data artifact contains the paper PDF, the source of this paper, the test vectors, and `SHA256SUMS`. The research kernel, the test harness, the extractors, and the Lean sources are not included in the artifact.

component	recorded evidence in this artifact
Lean (Mathlib-free) core, arrangement, comparator, DCEL, outcome, checks	`data/artifacts/test-vectors/s4_fuzz_expected.json`
Lean (Mathlib) order and cycle partition	`data/artifacts/test-vectors/s4_fuzz_expected.json`
Full-fuzz refinement counts	`data/artifacts/test-vectors/s4_fuzz_expected.json`
Rust fuzz, adversarial, and scaling counts	`data/artifacts/test-vectors/s4_fuzz_expected.json`
Paper PDF	`papers/S4-companion.pdf`, `SHA256SUMS`

All numeric results in this paper are the values recorded in the artifact data. Timing values are host-dependent and are not part of the reproducibility claim.

References

[1] A. A. G. Requicha and H. B. Voelcker. Solid modeling: a historical summary and contemporary assessment. *IEEE Computer Graphics and Applications*, 2(2):9–24, 1982.

[2] A. A. G. Requicha and H. B. Voelcker. Solid modeling: current status and research directions. *IEEE Computer Graphics and Applications*, 3(7):25–37, 1983. DOI: 10.1109/MCG.1983.263271.

[3] R. B. Tilove. Set membership classification: a unified approach to geometric intersection problems. *IEEE Transactions on Computers*, C-29(10):874–883, 1980.

[4] J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997. DOI: 10.1007/PL00009321.

[5] E. Fogel, D. Halperin, and R. Wein. *CGAL Arrangements and Their Applications: A Step-by-Step Guide*. Geometry and Computing 7, Springer, 2012. DOI: 10.1007/978-3-642-17283-0.

[6] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, Springer, 2021. DOI: 10.1007/978-3-030-79876-5_37.

[7] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. DOI: 10.1145/3372885.3373824.

[8] S. Ho and J. Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, 6(ICFP), Article 116, pages 711–741, 2022. DOI: 10.1145/3547647.

[9] P. M. M. de Castro, F. Cazals, S. Loriot, and M. Teillaud. CGAL 3D spherical geometry kernel. User Manual, CGAL. Package documentation.

[10] H. Bieri. Nef polyhedra: a brief introduction. In *Geometric Modelling*, Computing Supplement 10, pages 43–60, Springer, 1995. DOI: 10.1007/978-3-7091-7584-2_3.

[11] J. Keyser, T. Culver, M. Foskey, S. Krishnan, and D. Manocha. ESOLID—a system for exact boundary evaluation. In *Proceedings of the 7th ACM Symposium on Solid Modeling and Applications (SMA 2002)*, pages 23–34, 2002. DOI: 10.1145/566282.566289.

[12] S5 companion paper. *Sound and Deterministic Quadric Arrangements on a Rational Ellipsoid with Machine-Checkable Certificates*. Companion preprint, 2026.